

UNIVERSIDAD PERUANA
CAYETANO HEREDIA

FACULTAD DE CIENCIAS E INGENIERÍA

CURSO:

SISTEMAS OPERATIVOS

PROYECTO FINAL

INTEGRANTES

Diego Murga

Saori Jimenez

Anderson Delerna

Cesar Milla

DOCENTE:
Roberto Burga

2026

1. Introducción

La administración de memoria es una de las funciones fundamentales de un sistema operativo, ya que permite distribuir eficientemente los recursos de almacenamiento temporal entre los procesos que se encuentran en ejecución. Una gestión adecuada de la memoria contribuye a mejorar el rendimiento del sistema, garantizar la estabilidad de las aplicaciones y aprovechar los recursos disponibles.

En Linux, la administración de memoria comprende mecanismos como la asignación de memoria física, la memoria virtual, la paginación, el almacenamiento en caché y el uso de espacio de intercambio o *swap*. Estos mecanismos permiten que los procesos trabajen con espacios de direcciones virtuales y que el sistema operativo administre la memoria física de acuerdo con las necesidades de ejecución.

El presente proyecto tiene como finalidad analizar experimentalmente el comportamiento de la memoria principal y la memoria virtual en Linux. Para ello, se utilizarán herramientas de monitoreo como free, vmstat, top y htop, que permiten obtener información sobre el consumo de memoria, los procesos activos y el intercambio de páginas.

Asimismo, se desarrollará una simulación controlada de alta carga de memoria mediante Python, con el propósito de observar las variaciones en el consumo de RAM y estudiar el comportamiento de los mecanismos de paginación.

Los resultados permitirán comprender la relación entre la utilización de memoria, la disponibilidad de recursos y el rendimiento del sistema operativo

2. Objetivos

2.1. Objetivo general:

Analizar el funcionamiento de la administración de memoria principal y memoria virtual en Linux mediante herramientas de monitoreo y experimentos controlados de consumo de memoria.

2.2. Objetivos específicos:

-Identificar la cantidad de memoria RAM total, utilizada y disponible mediante herramientas de monitoreo de Linux.

-Analizar el uso de memoria swap y su relación con la administración de memoria virtual.

-Examinar el consumo de memoria de los procesos activos utilizando los comandos top y htop.

-Simular una carga progresiva de memoria mediante un programa desarrollado en Python.

-Analizar el comportamiento de la paginación a través de los contadores de fallos de página e intercambio de memoria.

-Comparar los resultados obtenidos antes, durante y después de la simulación para evaluar el comportamiento del sistema.

3. Marco teórico:

3.1. Administración de memoria en Linux

La administración de memoria es una de las funciones fundamentales de los sistemas operativos, ya que permite asignar, gestionar y liberar los recursos de memoria utilizados por los procesos en ejecución.

Linux emplea diferentes mecanismos para administrar eficientemente la memoria, entre ellos la memoria virtual, la paginación, el almacenamiento en caché y el espacio de intercambio o swap.

Estos mecanismos permiten ejecutar múltiples procesos simultáneamente, proteger sus espacios de direcciones y aprovechar los recursos físicos disponibles.

3.2. Memoria RAM

La memoria de acceso aleatorio (RAM) es un componente físico que almacena temporalmente los datos e instrucciones necesarios para la ejecución de programas.

Linux distribuye la memoria RAM entre los procesos, el kernel y las estructuras de almacenamiento en caché.

Para analizar su utilización se consideran los siguientes indicadores:

- Memoria total: cantidad de RAM reconocida por el sistema operativo.
- Memoria utilizada: memoria ocupada por procesos y componentes del sistema.
- Memoria libre: memoria que no se encuentra utilizada.
- Memoria disponible: estimación de memoria que puede asignarse a nuevas aplicaciones sin necesidad de utilizar swap.
- Búferes y caché: memoria utilizada para almacenar temporalmente información y mejorar el acceso a datos.

3.3. Memoria virtual

La memoria virtual es un mecanismo mediante el cual el sistema operativo proporciona a cada proceso un espacio de direcciones virtuales.

Estas direcciones se traducen a direcciones físicas mediante mecanismos de hardware y tablas de páginas administradas por el sistema operativo.

Entre sus beneficios se encuentran el aislamiento de procesos, la protección de memoria y la administración dinámica de páginas.

3.4. Memoria swap

La swap es un espacio de intercambio que Linux puede utilizar para almacenar temporalmente determinadas páginas de memoria que no necesitan permanecer en RAM.

Este mecanismo puede contribuir a liberar memoria física cuando existe presión de memoria.

Sin embargo, el intercambio frecuente con dispositivos de almacenamiento puede afectar el rendimiento, debido a que el acceso a estos dispositivos suele ser más lento que el acceso a RAM.

Es importante señalar que la existencia de memoria virtual no depende de que la swap esté habilitada.

3.5. Paginación

La paginación es una técnica de administración de memoria que divide los espacios de direcciones virtuales en unidades denominadas páginas.

Cuando un proceso accede a una página que requiere intervención del kernel, se produce un fallo de página.

Los fallos de página se clasifican principalmente en:

- Fallos menores: se resuelven sin necesidad de realizar lecturas desde almacenamiento.
- Fallos mayores: requieren operaciones de entrada/salida para obtener los datos necesarios.

Por lo tanto, los fallos de página pueden producirse incluso cuando no existe memoria swap activa.

3.6. Herramientas de monitoreo

Para el desarrollo experimental se utilizaron las siguientes herramientas:

- free: muestra la memoria RAM total, utilizada, libre y disponible, así como la memoria swap.
- vmstat: proporciona estadísticas sobre procesos, memoria, intercambio, operaciones de entrada/salida y CPU.
- top: permite observar los procesos activos y su consumo de recursos.

- htop: ofrece una interfaz interactiva para visualizar procesos y utilización de memoria y CPU.
- /proc/vmstat: proporciona contadores del kernel relacionados con la administración de memoria y la paginación.

4. Entorno experimental

Para el desarrollo del proyecto se utilizó el sistema operativo Ubuntu Linux, en el cual se ejecutaron diferentes comandos y herramientas para analizar la administración de memoria principal y memoria virtual.

Se emplearon los comandos free, vmstat, top y htop para monitorear el consumo de recursos. Asimismo, se desarrolló un programa en Python que permitió simular una carga progresiva de memoria RAM.

Los experimentos se realizaron en condiciones normales de funcionamiento, utilizando aplicaciones y procesos activos del sistema.

4.1. Características del entorno

Tabla 1. Características de memoria del sistema

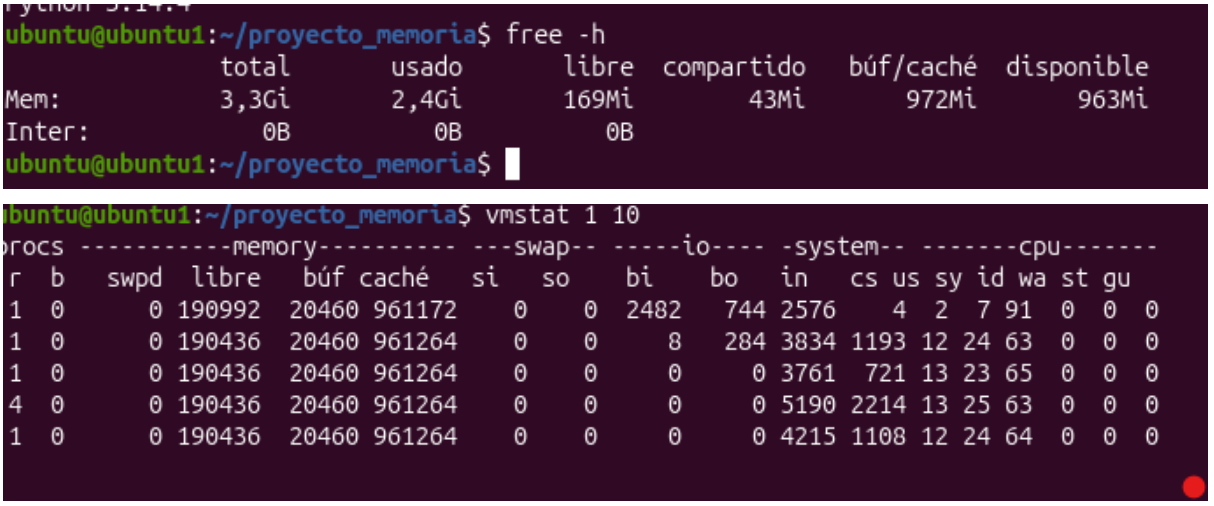
Característica	Resultado
Sistema operativo	Ubuntu Linux
Memoria RAM total	Aproximadamente 3,3 GiB
Memoria swap total	0 GiB
Memoria swap utilizada	0 GiB
Parámetro swappiness	60
Lenguaje utilizado	Python 3
Herramientas de monitoreo	free, vmstat, top y htop

Los resultados obtenidos mediante free -h permitieron identificar que el sistema disponía de aproximadamente 3,3 GiB de memoria RAM.

Por otro lado, el comando swapon --show no presentó dispositivos ni archivos de intercambio activos. Esta observación se confirmó mediante los resultados de free -h, que mostraron una capacidad swap de 0 GiB.

El parámetro vm.swappiness presentó un valor de 60. Sin embargo, este parámetro no implica que exista espacio swap disponible, sino que influye en las decisiones de recuperación de memoria cuando el sistema dispone de mecanismos de intercambio.

Figura 1. Características generales de memoria del sistema Ubuntu.



5. Desarrollo experimental

5.1. Experimento 1: análisis inicial de memoria

Objetivo: identificar la distribución de memoria RAM, la disponibilidad de espacio swap y los procesos que consumen recursos en Linux.

Procedimiento:

En primer lugar, se ejecutó el comando free -m para obtener información sobre la memoria total, utilizada, libre y disponible.

Posteriormente, se utilizó el comando vmstat 1 para monitorear las estadísticas de memoria, procesos, intercambio y actividad del procesador.

A continuación, se ejecutó top para identificar los procesos activos y ordenarlos según su consumo de memoria.

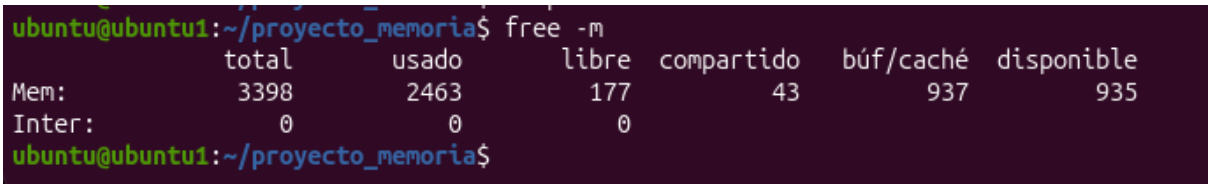
Finalmente, se utilizó htop para visualizar de manera interactiva el consumo de RAM, swap y CPU.

5.1.1. Resultados del comando free

Tabla 2. Estado inicial de la memoria

Indicador	Resultado
RAM total	3398 MiB
RAM utilizada	2463 MiB
RAM libre	177 MiB
Búferes y caché	937 MiB
RAM disponible	935 MiB
Swap total	0 MiB
Swap utilizada	0 MiB

Figura 2. Distribución inicial de memoria RAM y swap mediante el comando free.



Análisis:

Los resultados mostraron que el sistema Ubuntu disponía de aproximadamente 3398 MiB de memoria RAM, de los cuales 2463 MiB se encontraban utilizados.

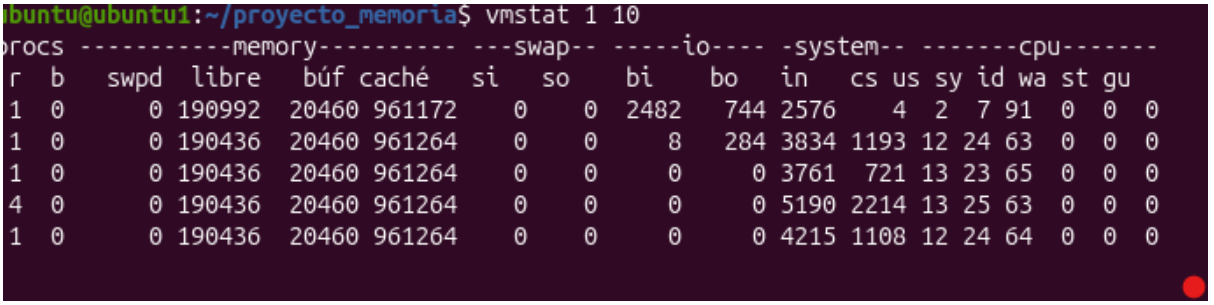
Asimismo, se registraron 177 MiB de memoria completamente libre y 935 MiB de memoria disponible.

La diferencia entre la memoria libre y disponible se relaciona con la capacidad de Linux para recuperar parte de la memoria utilizada en caché y otras estructuras del sistema.

En cuanto a la memoria swap, se observó que su capacidad total y utilizada era de 0 MiB, indicando que no existía espacio de intercambio activo.

5.1.2. Resultados del comando vmstat

Figura 3. Monitoreo de memoria e intercambio mediante vmstat.



Análisis:

Durante el monitoreo mediante vmstat, se observaron variaciones en los indicadores de memoria libre, búferes y caché.

Las columnas si y so permanecieron en cero durante las mediciones observadas, indicando que no se registraron transferencias de memoria hacia o desde swap.

Asimismo, se identificó actividad de entrada/salida y utilización del procesador, lo que evidencia que el sistema mantenía diferentes procesos activos durante el monitoreo.

5.1.3. Resultados de top y htop

Figura 4. Procesos ordenados por consumo de memoria mediante top.

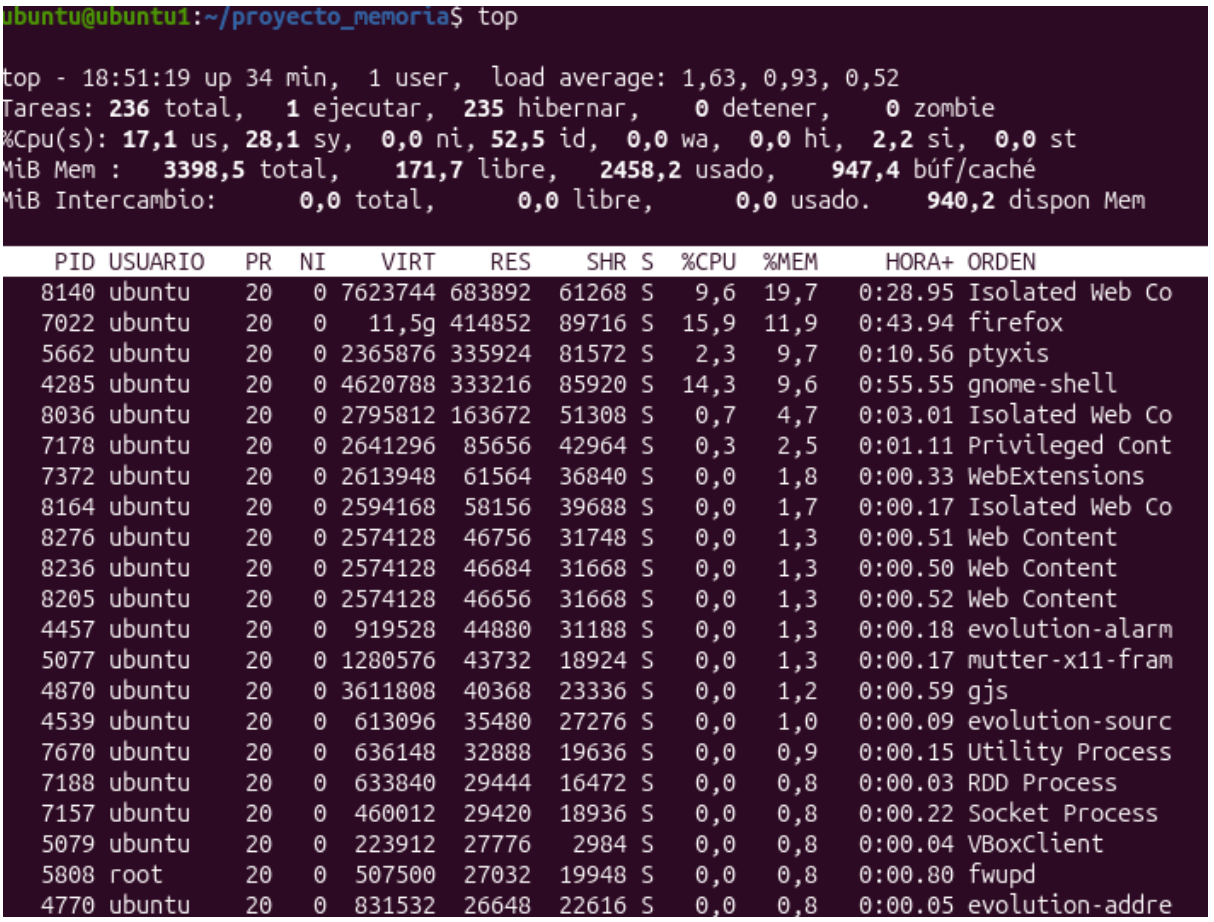
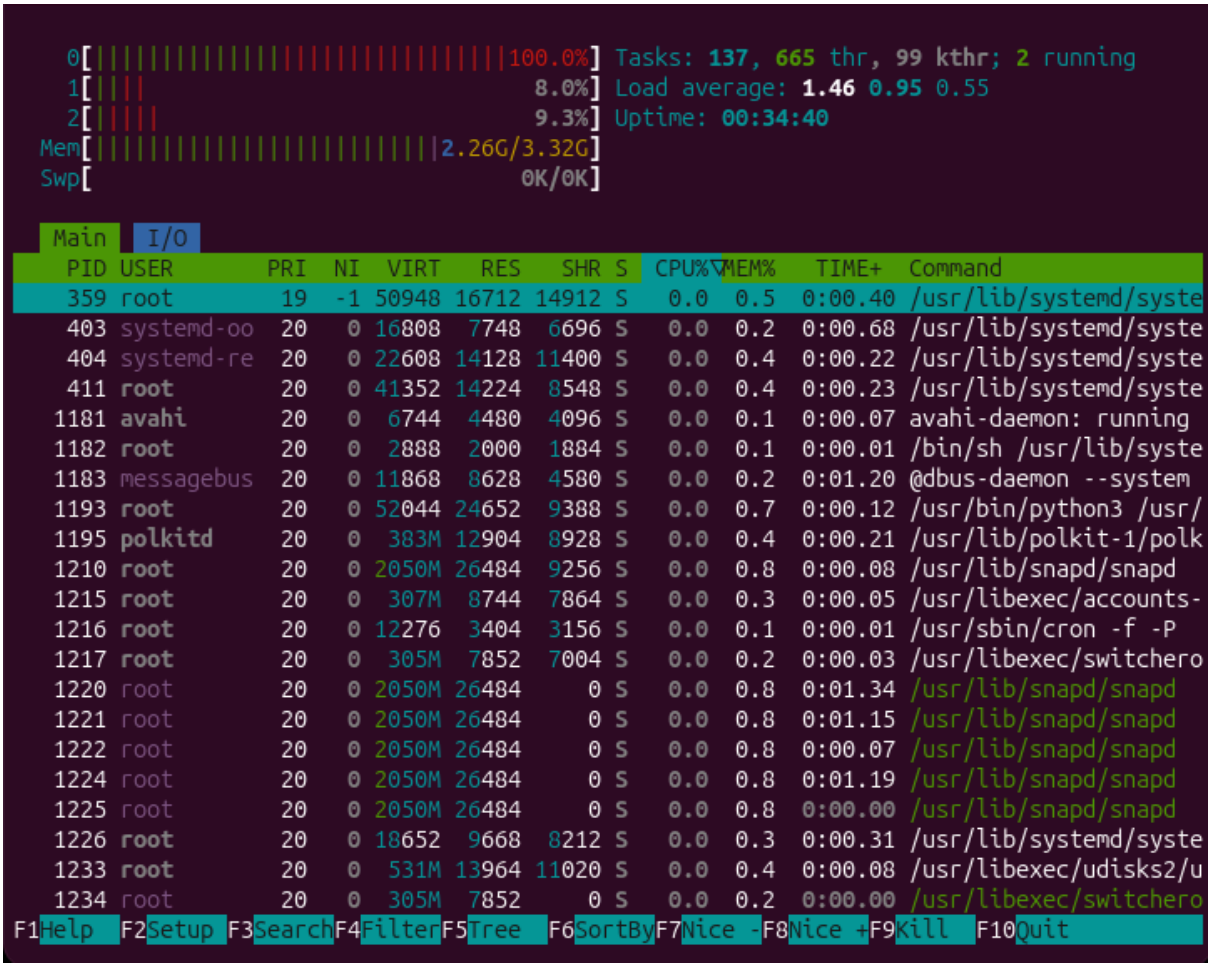


Figura 5. Monitoreo interactivo de procesos mediante htop.



Análisis:

Las herramientas top y htop permitieron identificar los procesos que consumían mayor cantidad de memoria RAM.

En la captura de top, el proceso identificado como Isolated Web Co presentó un consumo aproximado de 19,7 % de memoria, mientras que Firefox registró aproximadamente 11,9 %.

Estos resultados muestran que los procesos relacionados con la navegación web representaban una parte importante del consumo de memoria del sistema.

Asimismo, htop permitió visualizar gráficamente la utilización de CPU, RAM y swap, facilitando la identificación de los procesos activos y su consumo de recursos.

En conjunto, las herramientas utilizadas permitieron conocer el estado inicial de la memoria y comprobar que el sistema no realizaba operaciones de intercambio con swap durante las mediciones.

5.2. Experimento 2: simulación de alta carga de memoria

Objetivo: analizar el comportamiento de la administración de memoria de Linux ante un incremento controlado del consumo de RAM.

5.2.1. Desarrollo de la simulación

Para realizar el experimento se desarrolló un programa en Python denominado simulacion_memoria.py.

El programa utiliza una lista para almacenar bloques de memoria de 50 MB, incrementando progresivamente la cantidad reservada hasta alcanzar aproximadamente 500 MB.

Cada bloque se inicializa mediante operaciones de escritura, con la finalidad de provocar el uso efectivo de páginas de memoria.

Entre cada reserva se establece una pausa de tres segundos, permitiendo observar el comportamiento de los recursos mediante herramientas de monitoreo.

Una vez alcanzados los 500 MB, el programa mantiene los bloques reservados durante 30 segundos y posteriormente libera las referencias almacenadas.

Figura 6. Código fuente del programa de simulación de memoria.

[INSERTAR CAPTURA DEL CÓDIGO PYTHON]

5.2.2. Ejecución del programa

Durante la ejecución, el programa mostró incrementos progresivos de memoria reservada:

Etapa	Memoria reservada
1	50 MB
2	100 MB
3	150 MB
4	200 MB
5	250 MB
6	300 MB
7	350 MB
8	400 MB
9	450 MB
10	500 MB

El programa completó correctamente la secuencia de reservas hasta alcanzar aproximadamente 500 MB.

Figura 7. Ejecución del programa Python con reserva progresiva de memoria.

[INSERTAR CAPTURA DEL PROGRAMA EJECUTÁNDOSE]

5.2.3. Resultados del monitoreo

Se realizaron mediciones mediante el comando free -h antes, durante y después de la ejecución.

Tabla 3. Comparación del consumo de memoria

Indicador	Antes	Durante	Después
RAM total	3,3 GiB	3,3 GiB	3,3 GiB
RAM utilizada	2,3 GiB	2,3 GiB	2,4 GiB
RAM libre	212 MiB	702 MiB	609 MiB
RAM disponible	1,0 GiB	1,0 GiB	991 MiB
Swap utilizada	0 GiB	0 GiB	0 GiB

Figura 8. Estado de la memoria antes de la simulación.

[INSERTAR CAPTURA DE MEMORIA_ANTES]

Figura 9. Estado de la memoria durante la simulación.

[INSERTAR CAPTURA DE MEMORIA_DURANTE]

Figura 10. Estado de la memoria después de la simulación.

[INSERTAR CAPTURA DE MEMORIA_DESPUÉS]

5.2.4. Análisis de resultados

El programa desarrollado en Python permitió realizar una reserva progresiva de memoria hasta alcanzar aproximadamente 500 MB.

Durante las mediciones se observaron variaciones en la distribución de memoria del sistema.

Antes de la simulación, la memoria utilizada era de aproximadamente 2,3 GiB, mientras que la memoria libre era de 212 MiB.

Durante la medición identificada como carga, la memoria utilizada se mantuvo alrededor de 2,3 GiB, mientras que la memoria libre registrada fue de 702 MiB.

Después de la simulación, la memoria utilizada fue de aproximadamente 2,4 GiB y la memoria libre de 609 MiB.

Aunque el programa completó correctamente la reserva de memoria, las mediciones globales no mostraron un incremento sostenido claramente atribuible a la simulación.

Esto puede relacionarse con la actividad simultánea de otros procesos, la recuperación de memoria caché y el momento exacto en que se realizaron las mediciones.

Por lo tanto, no es posible atribuir todas las variaciones observadas únicamente al programa Python.

En relación con la memoria swap, su utilización permaneció en cero durante las tres mediciones, debido a que el sistema no tenía espacio de intercambio activo.

El experimento permitió observar la ejecución de una aplicación que incrementa progresivamente sus necesidades de memoria y analizar la importancia de realizar mediciones sincronizadas para evaluar el consumo real de recursos.

5.2. Experimento 2: simulación de alta carga de memoria

Objetivo: analizar el comportamiento de la administración de memoria de Linux ante un incremento controlado del consumo de RAM.

5.2.1. Desarrollo de la simulación

Para realizar el experimento se desarrolló un programa en Python denominado simulacion_memoria.py.

El programa utiliza una lista para almacenar bloques de memoria de 50 MB, incrementando progresivamente la cantidad reservada hasta alcanzar aproximadamente 500 MB.

Cada bloque se inicializa mediante operaciones de escritura, con la finalidad de provocar el uso efectivo de páginas de memoria.

Entre cada reserva se establece una pausa de tres segundos, permitiendo observar el comportamiento de los recursos mediante herramientas de monitoreo.

Una vez alcanzados los 500 MB, el programa mantiene los bloques reservados durante 30 segundos y posteriormente libera las referencias almacenadas.

Figura 6. Código fuente del programa de simulación de memoria.

```
import time

bloques = []
tamano_mb = 50
limite_mb = 500

print("SIMULACION DE CARGA DE MEMORIA")

try:
        for i in range(limite_mb // tamano_mb):
                bloque = bytearray(tamano_mb * 1024 * 1024)

                for j in range(0, len(bloque), 4096):
                        bloque[j] = 1

                bloques.append(bloque)
                memoria = len(bloques) * tamano_mb
                print(f"Memoria reservada: {memoria} MB")

                time.sleep(3)
        print("Manteniendo la carga durante 30 segundos")
        time.sleep(30)

except (KeyboardInterrupt, MemoryError):
        print("Simulacion interrumpida")

finally:
        bloques.clear()
        print("Memoria liberada")
```

5.2.2. Ejecución del programa

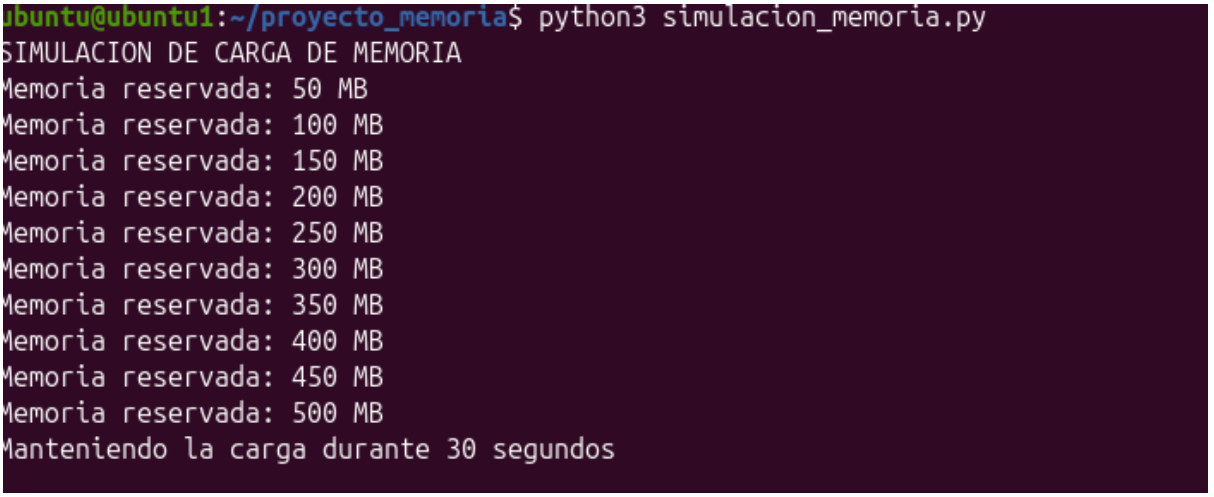
Durante la ejecución, el programa mostró incrementos progresivos de memoria reservada:

Etapa	Memoria reservada
1	50 MB
2	100 MB
3	150 MB
4	200 MB
5	250 MB
6	300 MB
7	350 MB
8	400 MB
9	450 MB

10	500 MB

El programa completó correctamente la secuencia de reservas hasta alcanzar aproximadamente 500 MB.

Figura 7. Ejecución del programa Python con reserva progresiva de memoria.



5.2.3. Resultados del monitoreo

Se realizaron mediciones mediante el comando free -h antes, durante y después de la ejecución.

Tabla 3. Comparación del consumo de memoria

Indicador	Antes	Durante	Después
RAM total	3,3 GiB	3,3 GiB	3,3 GiB
RAM utilizada	2,3 GiB	2,3 GiB	2,4 GiB
RAM libre	212 MiB	702 MiB	609 MiB
RAM disponible	1,0 GiB	1,0 GiB	991 MiB
Swap utilizada	0 GiB	0 GiB	0 GiB

Figura 8. Estado de la memoria antes de la simulación.

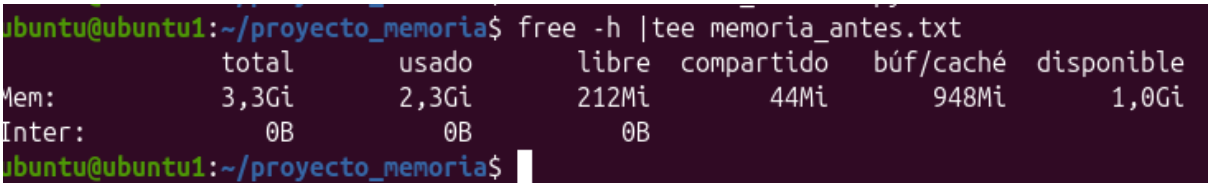


Figura 9. Estado de la memoria durante la simulación.

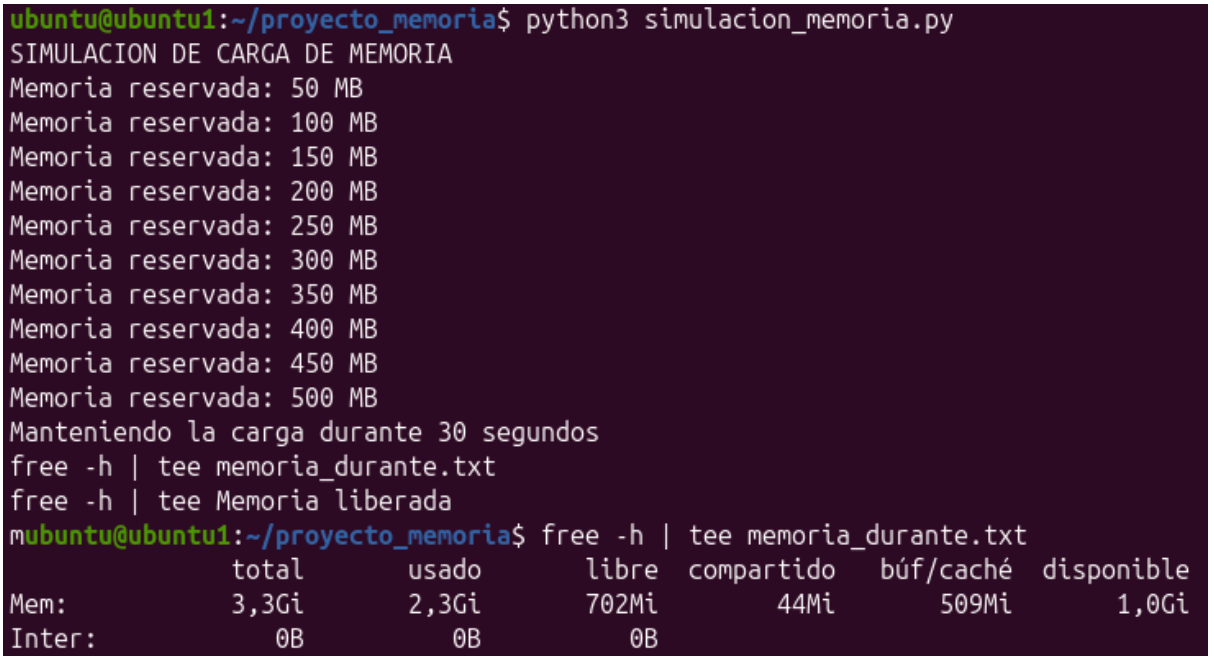
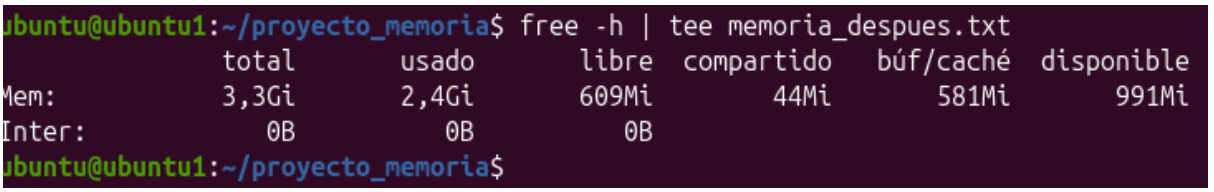


Figura 10. Estado de la memoria después de la simulación.



5.2.4. Análisis de resultados

El programa desarrollado en Python permitió realizar una reserva progresiva de memoria hasta alcanzar aproximadamente 500 MB.

Durante las mediciones se observaron variaciones en la distribución de memoria del sistema.

Antes de la simulación, la memoria utilizada era de aproximadamente 2,3 GiB, mientras que la memoria libre era de 212 MiB.

Durante la medición identificada como carga, la memoria utilizada se mantuvo alrededor de 2,3 GiB, mientras que la memoria libre registrada fue de 702 MiB.

Después de la simulación, la memoria utilizada fue de aproximadamente 2,4 GiB y la memoria libre de 609 MiB.

Aunque el programa completó correctamente la reserva de memoria, las mediciones globales no mostraron un incremento sostenido claramente atribuible a la simulación.

Esto puede relacionarse con la actividad simultánea de otros procesos, la recuperación de memoria caché y el momento exacto en que se realizaron las mediciones.

Por lo tanto, no es posible atribuir todas las variaciones observadas únicamente al programa Python.

En relación con la memoria swap, su utilización permaneció en cero durante las tres mediciones, debido a que el sistema no tenía espacio de intercambio activo.

El experimento permitió observar la ejecución de una aplicación que incrementa progresivamente sus necesidades de memoria y analizar la importancia de realizar mediciones sincronizadas para evaluar el consumo real de recursos.

5.3. Experimento 3: análisis de paginación y memoria virtual

Objetivo: analizar el comportamiento de los mecanismos de paginación de Linux mediante los contadores del kernel y determinar si se producen operaciones de intercambio con swap.

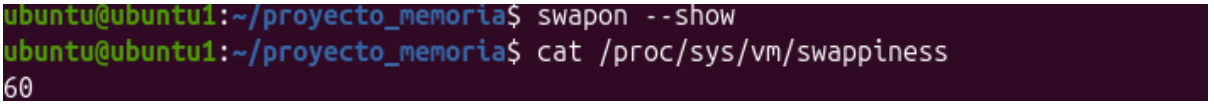
5.3.1. Verificación de la configuración de swap

En primer lugar, se ejecutó el comando swapon --show para verificar la existencia de dispositivos o archivos de intercambio activos.

El comando no mostró resultados, indicando que no existía espacio swap habilitado.

Posteriormente, se consultó el parámetro vm.swappiness, obteniéndose un valor de 60.

Figura 11. Verificación de la configuración de swap y del parámetro swappiness.



5.3.2. Registro de contadores de paginación

Se utilizó el archivo /proc/vmstat para obtener información sobre los siguientes indicadores:

- pgfault: número acumulado de fallos de página.
- pgmajfault: número acumulado de fallos mayores.
- pswpin: páginas recuperadas desde swap.
- pswpout: páginas enviadas hacia swap.

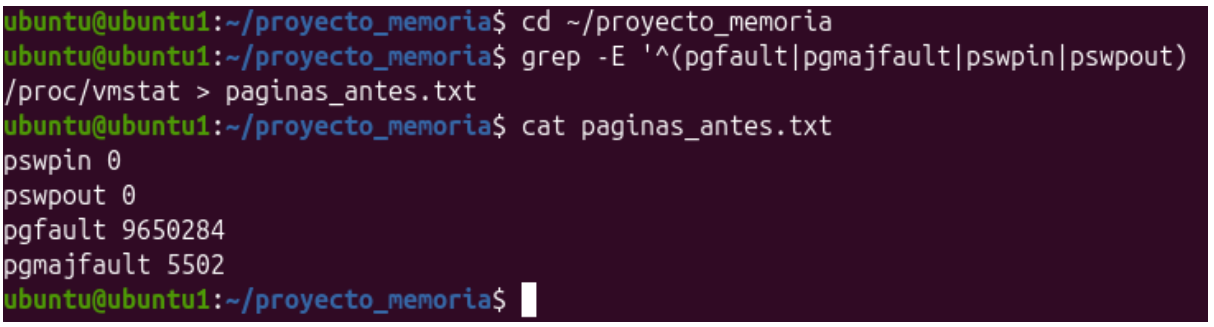
Inicialmente, se registraron los contadores antes de ejecutar nuevamente la simulación de memoria.

Tabla 4. Contadores iniciales de paginación

Indicador	Valor inicial

pgfault	9 650 284
pgmajfault	5 502
pswpin	0
pswpout	0

Figura 12. Contadores iniciales de paginación obtenidos mediante /proc/vmstat.



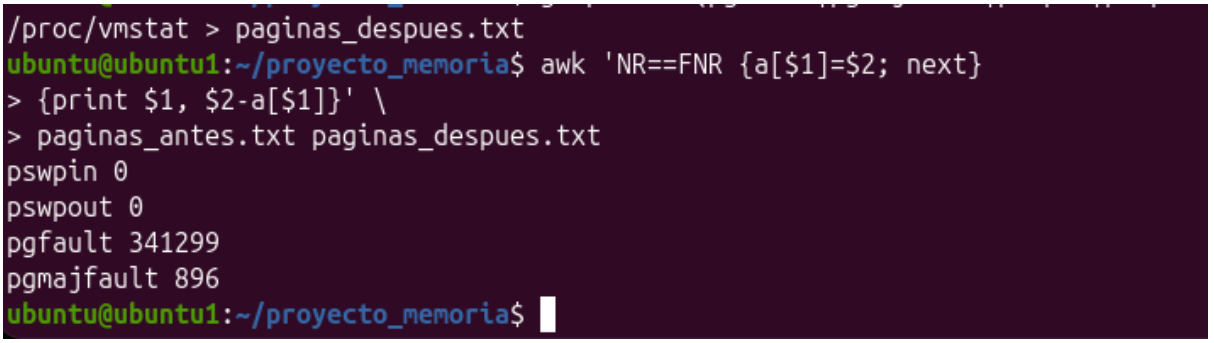
5.3.3. Comparación de resultados

Después de ejecutar la simulación, se registraron nuevamente los contadores y se calcularon sus diferencias mediante el comando awk.

Tabla 5. Variación de los contadores de paginación

Indicador	Valor inicial	Variación
pgfault	9 650 284	+341 299
pgmajfault	5 502	+896
pswpin	0	0
pswpout	0	0

Figura 13. Diferencias de los contadores de paginación después de la simulación.



5.3.4. Análisis de resultados

Durante el intervalo de medición se registraron 341 299 fallos de página adicionales y 896 fallos mayores.

Estos resultados evidencian actividad en los mecanismos de paginación del sistema operativo.

Los fallos de página pueden producirse cuando un proceso necesita acceder a una página que requiere intervención del kernel. En algunos casos, el sistema puede resolverlos sin acceder al almacenamiento; en otros, se requieren operaciones de entrada/salida.

Por otro lado, los indicadores pswpin y pswpout permanecieron en cero, lo que demuestra que no se registraron operaciones de intercambio de páginas con swap durante el intervalo observado.

Este resultado es coherente con la ausencia de espacio swap activo en el sistema.

Es importante considerar que los contadores de /proc/vmstat corresponden a la actividad global del sistema y no exclusivamente al programa de simulación.

En consecuencia, los resultados permiten identificar actividad de paginación, pero no permiten atribuir todos los fallos registrados únicamente al programa Python.

El experimento demuestra que la paginación es un mecanismo independiente de la utilización de swap y que Linux puede registrar fallos de página incluso cuando no dispone de espacio de intercambio habilitado.

6. Resultados y Análisis

Los tres experimentos realizados permitieron analizar diferentes aspectos de la administración de memoria principal y memoria virtual en Ubuntu Linux.

En el primer experimento se identificó la distribución inicial de memoria RAM, registrándose una capacidad aproximada de 3,3 GiB. Asimismo, se observó que una parte importante de la memoria era utilizada por procesos activos, principalmente aplicaciones relacionadas con la navegación web.

En el segundo experimento se desarrolló una simulación mediante Python, que permitió reservar progresivamente hasta 500 MB de memoria.

Las mediciones realizadas antes, durante y después de la ejecución mostraron variaciones en los indicadores de memoria utilizada, libre y disponible. Sin embargo, los resultados no permitieron establecer una relación directa entre todas las variaciones observadas y la ejecución del programa, debido a la actividad simultánea de otros procesos.

En el tercer experimento se analizaron los mecanismos de paginación mediante los contadores del kernel. Se registraron 341 299 fallos de página y 896 fallos mayores adicionales durante el intervalo estudiado.

Por otro lado, no se observaron operaciones de intercambio con swap, debido a que el sistema no contaba con espacio de intercambio activo.

6.1. Comparación de los experimentos

Tabla 6. Resumen de resultados experimentales

Aspecto	Resultado
RAM total del sistema	3,3 GiB
Memoria utilizada inicialmente	2463 MiB
Memoria disponible inicialmente	935 MiB
Memoria máxima reservada por Python	500 MB
Fallos de página adicionales	341 299
Fallos mayores adicionales	896
Operaciones de entrada desde swap	0
Operaciones de salida hacia swap	0
Espacio swap activo	No disponible

6.2. Evaluación del rendimiento

El monitoreo permitió comprobar que Linux administra la memoria mediante diferentes mecanismos que incluyen la asignación de recursos, el almacenamiento en caché y la paginación.

La ejecución de la simulación demostró que una aplicación puede solicitar y mantener bloques de memoria durante un periodo determinado.

Sin embargo, debido a la actividad simultánea de otros procesos, las mediciones globales no permitieron cuantificar de forma aislada el impacto del programa sobre el consumo total de RAM.

Por otro lado, la ausencia de espacio swap activo permitió observar que los fallos de página no dependen necesariamente de operaciones de intercambio.

En términos generales, los resultados permitieron comprender el comportamiento de la memoria principal y virtual, así como la importancia de utilizar herramientas de monitoreo para interpretar adecuadamente el rendimiento del sistema operativo.

7. Conclusiones

1. Se logró analizar la administración de memoria en Ubuntu Linux mediante las herramientas free, vmstat, top y htop, identificando la distribución de memoria RAM y el consumo de recursos de los procesos activos.
2. El sistema presentó una capacidad aproximada de 3,3 GiB de RAM. Durante una de las mediciones iniciales se registraron 2463 MiB de memoria utilizada y 935 MiB disponibles para nuevas aplicaciones.
3. La simulación desarrollada en Python permitió reservar progresivamente hasta 500 MB de memoria, demostrando el funcionamiento de una aplicación que incrementa sus requerimientos de recursos durante la ejecución.
4. El análisis de los contadores de paginación registró 341 299 fallos de página y 896 fallos mayores adicionales, evidenciando actividad en los mecanismos de administración de memoria virtual.
5. No se registraron operaciones de intercambio con swap, debido a que el entorno experimental no disponía de espacio swap activo. Esto permitió comprobar que los fallos de página pueden producirse independientemente del uso de swap.
6. Finalmente, se concluye que las herramientas de monitoreo de Linux permiten estudiar la administración de memoria y comprender la relación entre la utilización de RAM, la ejecución de procesos y los mecanismos de memoria virtual.

8. RECOMENDACIONES

-Realizar mediciones de memoria antes, durante y después de una carga controlada, procurando mantener condiciones similares de ejecución.

-Utilizar herramientas como htop y vmstat para monitorear el comportamiento de los recursos en tiempo real.

-Considerar la actividad de otros procesos al interpretar las variaciones en el consumo global de memoria.

-Para futuras pruebas, habilitar un espacio swap de forma controlada y comparar su comportamiento con el de un sistema sin intercambio activo.

-Complementar los experimentos con mediciones específicas del proceso evaluado, con la finalidad de determinar su consumo de memoria de manera más precisa.

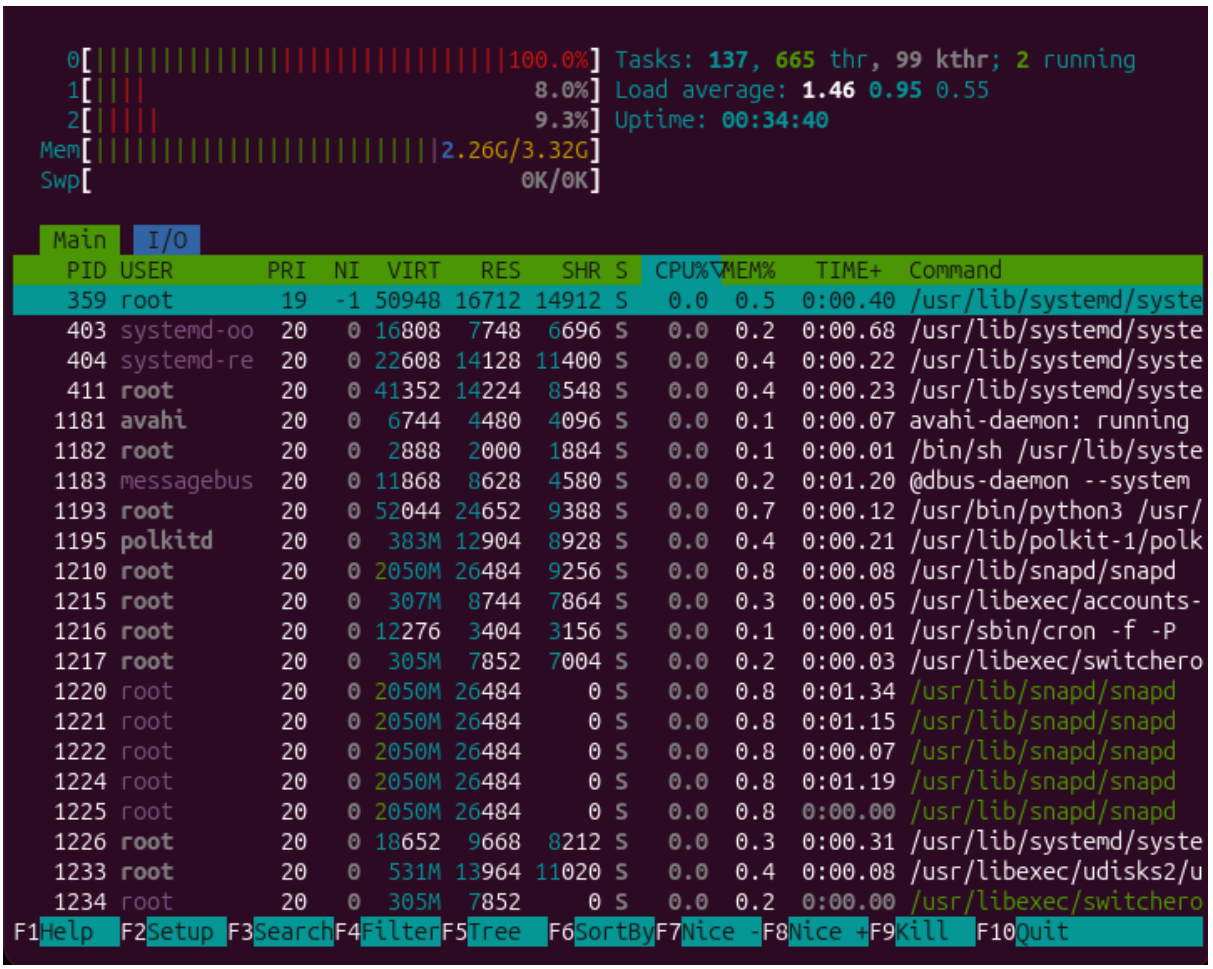
```
ubuntu@ubuntu1:~/proyecto_memoria$ free -h
               total        usado       libre  compartido   búf/caché  disponible
Mem:           3,3Gi       2,4Gi       169Mi        43Mi       972Mi       963Mi
Inter:            0B          0B          0B
ubuntu@ubuntu1:~/proyecto_memoria$
```

```
ubuntu@ubuntu1:~/proyecto_memoria$ vmstat 1 10
procs -----------memory---------- ---swap-- -----io---- -system-- -------cpu-------
 r  b   swpd  libre   búf caché   si   so    bi    bo   in   cs us sy id wa st gu
 1  0      0 190992  20460 961172    0    0  2482   744 2576    4  2  7 91  0  0  0
 1  0      0 190436  20460 961264    0    0     8   284 3834 1193 12 24 63  0  0  0
 1  0      0 190436  20460 961264    0    0     0     0 3761  721 13 23 65  0  0  0
 4  0      0 190436  20460 961264    0    0     0     0 5190 2214 13 25 63  0  0  0
 1  0      0 190436  20460 961264    0    0     0     0 4215 1108 12 24 64  0  0  0
```

```
ubuntu@ubuntu1:~/proyecto_memoria$ top

top - 18:51:19 up 34 min,  1 user,  load average: 1,63, 0,93, 0,52
Tareas: 236 total,   1 ejecutar,  235 hibernar,    0 detener,    0 zombie
%Cpu(s): 17,1 us, 28,1 sy,  0,0 ni, 52,5 id,  0,0 wa,  0,0 hi,  2,2 si,  0,0 st
MiB Mem :   3398,5 total,    171,7 libre,   2458,2 usado,    947,4 búf/caché
MiB Intercambio:      0,0 total,      0,0 libre,      0,0 usado.    940,2 dispon Mem

    PID USUARIO    PR  NI    VIRT    RES    SHR S  %CPU  %MEM     HORA+ ORDEN
   8140 ubuntu    20   0 7623744 683892  61268 S   9,6  19,7   0:28.95 Isolated Web Co
   7022 ubuntu    20   0   11,5g 414852  89716 S  15,9  11,9   0:43.94 firefox
   5662 ubuntu    20   0 2365876 335924  81572 S   2,3   9,7   0:10.56 ptyxis
   4285 ubuntu    20   0 4620788 333216  85920 S  14,3   9,6   0:55.55 gnome-shell
   8036 ubuntu    20   0 2795812 163672  51308 S   0,7   4,7   0:03.01 Isolated Web Co
   7178 ubuntu    20   0 2641296  85656  42964 S   0,3   2,5   0:01.11 Privileged Cont
   7372 ubuntu    20   0 2613948  61564  36840 S   0,0   1,8   0:00.33 WebExtensions
   8164 ubuntu    20   0 2594168  58156  39688 S   0,0   1,7   0:00.17 Isolated Web Co
   8276 ubuntu    20   0 2574128  46756  31748 S   0,0   1,3   0:00.51 Web Content
   8236 ubuntu    20   0 2574128  46684  31668 S   0,0   1,3   0:00.50 Web Content
   8205 ubuntu    20   0 2574128  46656  31668 S   0,0   1,3   0:00.52 Web Content
   4457 ubuntu    20   0  919528  44880  31188 S   0,0   1,3   0:00.18 evolution-alarm
   5077 ubuntu    20   0 1280576  43732  18924 S   0,0   1,3   0:00.17 mutter-x11-fram
   4870 ubuntu    20   0 3611808  40368  23336 S   0,0   1,2   0:00.59 gjs
   4539 ubuntu    20   0  613096  35480  27276 S   0,0   1,0   0:00.09 evolution-sourc
   7670 ubuntu    20   0  636148  32888  19636 S   0,0   0,9   0:00.15 Utility Process
   7188 ubuntu    20   0  633840  29444  16472 S   0,0   0,8   0:00.03 RDD Process
   7157 ubuntu    20   0  460012  29420  18936 S   0,0   0,8   0:00.22 Socket Process
   5079 ubuntu    20   0  223912  27776   2984 S   0,0   0,8   0:00.04 VBoxClient
   5808 root      20   0  507500  27032  19948 S   0,0   0,8   0:00.80 fwupd
   4770 ubuntu    20   0  831532  26648  22616 S   0,0   0,8   0:00.05 evolution-addre
```




```
ubuntu@ubuntu1:~/proyecto_memoria$ free -m
               total        usado       libre  compartido   búf/caché  disponible
Mem:            3398         2463          177          43         937         935
Inter:             0            0            0
ubuntu@ubuntu1:~/proyecto_memoria$
```

```
import time

bloques = []
tamano_mb = 50
limite_mb = 500

print("SIMULACION DE CARGA DE MEMORIA")

try:
        for i in range(limite_mb // tamano_mb):
                bloque = bytearray(tamano_mb * 1024 * 1024)

                for j in range(0, len(bloque), 4096):
                        bloque[j] = 1

                bloques.append(bloque)
                memoria = len(bloques) * tamano_mb
                print(f"Memoria reservada: {memoria} MB")

                time.sleep(3)
        print("Manteniendo la carga durante 30 segundos")
        time.sleep(30)

except (KeyboardInterrupt, MemoryError):
        print("Simulacion interrumpida")

finally:
        bloques.clear()
        print("Memoria liberada")
```

```
ubuntu@ubuntu1:~/proyecto_memoria$ free -h |tee memoria_antes.txt
               total        usado        libre  compartido   búf/caché  disponible
Mem:           3,3Gi        2,3Gi       212Mi        44Mi       948Mi       1,0Gi
Inter:            0B           0B           0B
ubuntu@ubuntu1:~/proyecto_memoria$
```

```
ubuntu@ubuntu1:~/proyecto_memoria$ python3 simulacion_memoria.py
SIMULACION DE CARGA DE MEMORIA
Memoria reservada: 50 MB
Memoria reservada: 100 MB
Memoria reservada: 150 MB
Memoria reservada: 200 MB
Memoria reservada: 250 MB
Memoria reservada: 300 MB
Memoria reservada: 350 MB
Memoria reservada: 400 MB
Memoria reservada: 450 MB
Memoria reservada: 500 MB
Manteniendo la carga durante 30 segundos
```

```
ubuntu@ubuntu1:~/proyecto_memoria$ vmstat 1
procs -----------memory---------- ---swap-- -----io---- -system-- -------cpu-------
 r  b   swpd  libre   búf caché   si   so    bi    bo   in   cs us sy id wa st gu
 5  0      0 147724   4456 609096    0    0  1818   511 2781    4  6 11 83  0  0  0
 1  0      0 172748   4568 566468    0    0 57975     0 4066 2484 14 26 59  1  0  0
 1  0      0 172632   4568 570964    0    0  4428     0 4680 2410 15 21 64  0  0  0
 3  0      0 162576   4568 582936    0    0 14153     0 4344 2707 15 22 63  0  0  0
 1  0      0 162508   4812 583020    0    0   244     0 3936 2098 14 21 64  1  0  0
 2  0      0 157808   4836 588304    0    0  5336     0 3784 1328 14 21 65  0  0  0
 1  0      0 156972   4836 589940    0    0  4178     0 3107 1075 14 20 65  0  0  0
 1  0      0 155908   4844 590820    0    0   792    36 3153  587 15 19 66  0  0  0
 1  0      0 155908   4844 590828    0    0     0     0 3227  473 13 21 66  0  0  0
 1  0      0 155908   4844 590828    0    0     0     0 3062  396 13 21 66  0  0  0
 1  0      0 155908   4844 590828    0    0     0     0 3081  307 14 20 67  0  0  0
 6  2      0 122808   4508 634716    0    0 57833     0 3396 2668 15 22 62  1  0  0
 1  0      0 182104   3316 555860    0    0 131895     0 4888 9564 17 33 49  0  0
0
 1  0      0 179588   3316 555740    0    0   507     0 4034 3638 17 23 60  0  0  0
 1  0      0 179588   3316 555800    0    0     0     0 2972  406 14 20 66  0  0  0
 1  0      0 179588   3316 555800    0    0     0     0 3063  287 14 20 66  0  0  0
 1  0      0 179588   3316 555800    0    0     0     0 3156  292 14 19 67  0  0  0
 1  0      0 179228   3316 556108    0    0  1446   260 2954  361 15 20 65  0  0  0
 1  0      0 179228   3316 556108    0    0   130     0 2984  292 13 20 67  0  0  0
 1  0      0 179168   3316 556812    0    0   716     0 3080  349 13 21 66  0  0  0
 1  0      0 177964   3316 557836    0    0  1301     0 2951  329 14 19 67  0  0  0
 1  0      0 177268   3320 558516    0    0  1549   284 2965  419 14 20 65  1  0  0
 1  0      0 177268   3320 558552    0    0     0     0 2844  220 14 20 66  0  0  0
 1  0      0 176488   3332 559540    0    0   468     0 2906  235 13 21 66  0  0  0
```

```
ubuntu@ubuntu1:~/proyecto_memoria$ python3 simulacion_memoria.py
SIMULACION DE CARGA DE MEMORIA
Memoria reservada: 50 MB
Memoria reservada: 100 MB
Memoria reservada: 150 MB
Memoria reservada: 200 MB
Memoria reservada: 250 MB
Memoria reservada: 300 MB
Memoria reservada: 350 MB
Memoria reservada: 400 MB
Memoria reservada: 450 MB
Memoria reservada: 500 MB
Manteniendo la carga durante 30 segundos
free -h | tee memoria_durante.txt
free -h | tee Memoria liberada
mubuntu@ubuntu1:~/proyecto_memoria$ free -h | tee memoria_durante.txt
               total        usado        libre  compartido   búf/caché  disponible
Mem:           3,3Gi        2,3Gi       702Mi        44Mi       509Mi       1,0Gi
Inter:            0B           0B           0B
```

```
ubuntu@ubuntu1:~/proyecto_memoria$ free -h | tee memoria_despues.txt
               total        usado        libre  compartido   búf/caché  disponible
Mem:           3,3Gi        2,4Gi       609Mi        44Mi       581Mi       991Mi
Inter:            0B           0B           0B
ubuntu@ubuntu1:~/proyecto_memoria$
```

```
ubuntu@ubuntu1:~/proyecto_memoria$ swapon --show
ubuntu@ubuntu1:~/proyecto_memoria$ cat /proc/sys/vm/swappiness
60
```

```
ubuntu@ubuntu1:~/proyecto_memoria$ cd ~/proyecto_memoria
ubuntu@ubuntu1:~/proyecto_memoria$ grep -E '^(pgfault|pgmajfault|pswpin|pswpout)
/proc/vmstat > paginas_antes.txt
ubuntu@ubuntu1:~/proyecto_memoria$ cat paginas_antes.txt
pswpin 0
pswpout 0
pgfault 9650284
pgmajfault 5502
ubuntu@ubuntu1:~/proyecto_memoria$
```

```
/proc/vmstat > paginas_despues.txt
ubuntu@ubuntu1:~/proyecto_memoria$ awk 'NR==FNR {a[$1]=$2; next}
> {print $1, $2-a[$1]}' \
> paginas_antes.txt paginas_despues.txt
pswpin 0
pswpout 0
pgfault 341299
pgmajfault 896
ubuntu@ubuntu1:~/proyecto_memoria$
```